

# WEB SYSTEMS DESIGN – PROJECT 12



**VISTULA**  
UNIVERSITY

**Name:** Amjad Massaoud

**Student ID:** 78709

**Course:** Web Systems Design

**Semester:** 6th semester 57DPH

**Date:** 30.08.2026

**GitHub:** <https://github.com/Jadtales/uni-projects>

This report documents the Project 12 extension of the Figma UI/UX design file. The work builds on the complete system architecture and adds API states, prototype flow, API/data mapping, developer notes, and design handoff evidence.

1. Introduction ..... 3

2. Objectives of Project 12 ..... 3

**3. Project 12 Implementation Summary ..... 3**

    3.1 API States ..... 3

    3.2 Prototype Flow ..... 3

    3.3 API and Data Mapping ..... 3

    3.4 Developer Notes and Handoff ..... 4

    3.5 Dev Mode and Naming Organization ..... 4

**4. Evidence Screenshots ..... 4**

    Figure 1. Figma file overview and cover page ..... 4

    Figure 2. Application map showing frontend, API, and storage architecture ..... 4

    Figure 3. Design system page showing reusable UI rules and tokens ..... 5

    Figure 4. Wireframes page showing low-fidelity module sketches ..... 6

    Figure 5. Final UI screens page showing the high-fidelity dashboard ..... 6

    Figure 6. Prototype flow page showing planned user navigation ..... 7

    Figure 7. API and data mapping table linking screens with endpoints ..... 8

    Figure 8. Developer notes page showing design handoff specifications ..... 9

    Figure 9. API states page showing loading, success, and error states ..... 9

5. Limitations and Future Improvements ..... 10

6. Conclusion ..... 10

7. References ..... 10

## 1. Introduction

Project 12 continues the Figma design documentation created for the existing web application. The system already includes a Laravel backend, Vue frontend, PostgreSQL database, Redis cache, Docker Compose deployment, file storage, and REST API endpoints. The goal of Project 12 is to improve the Figma file as a professional communication document for developers, designers, and stakeholders.

The Figma design represents implemented modules such as tasks, URL shortener, nearby restaurants, photo upload, news feed, video catalog, recommendations, watchlist, and continue watching.

## 2. Objectives of Project 12

- Add API-related interface states for loading, success, and error cases.
- Create or document prototype interactions between the main application screens.
- Improve the API and data mapping page so each UI action is connected to a backend endpoint.
- Prepare developer notes explaining reusable components, colors, typography, screen sizes, responsive behavior, API dependencies, limitations, and future improvements.
- Organize the Figma file using clear page names and readable design sections.
- Export evidence screenshots with captions for the final report.

## 3. Project 12 Implementation Summary

### 3.1 API States

The API States page documents how the interface should behave while data is loading, after a successful response, and when an error occurs. This helps the system handle real API behavior, not only the happy path.

| Module            | Loading                    | Success               | Error            |
|-------------------|----------------------------|-----------------------|------------------|
| Videos            | Loading...                 | Videos shown          | Failed           |
| Photos            | Uploading...               | Uploaded              | File too large   |
| Watchlist         | Saving...                  | Added                 | Duplicate        |
| Recommendations   | Loading recommendations... | Recommendations shown | Failed to load   |
| Continue Watching | Loading progress...        | Progress displayed    | Sync failed      |
| Tasks             | Saving task...             | Task saved            | Validation error |

### 3.2 Prototype Flow

The prototype flow begins from the dashboard and connects to the main modules. The design shows flows to task management, photo upload/gallery, news feed, video catalog, watchlist, recommendations, and continue-watching. This supports the requirement that the design should allow the main application flow to be understood visually.

### 3.3 API and Data Mapping

The API mapping page links the UI with Laravel REST API endpoints under the `/api/78709/v1` prefix.

### 3.4 Developer Notes and Handoff

The developer notes page explains the implementation relationship between Figma and the web application. It includes reusable components, colors, typography, screen sizes, responsive behavior, API dependencies, known limitations, and future improvements. This supports design handoff and makes the file understandable without extra explanation.

### 3.5 Dev Mode and Naming Organization

The Figma file uses clear page names and organized sections such as Application Map, Design System, Wireframes, Final UI Screens, Prototype Flow, API and Data Mapping, Developer Notes, and API States. In Figma, Dev Mode can be toggled with Shift + D to inspect dimensions, spacing, colors, typography, and code snippets.

## 4. Evidence Screenshots

The following screenshots are included as evidence. Captions are provided for each figure as required by the Project 12 instructions.

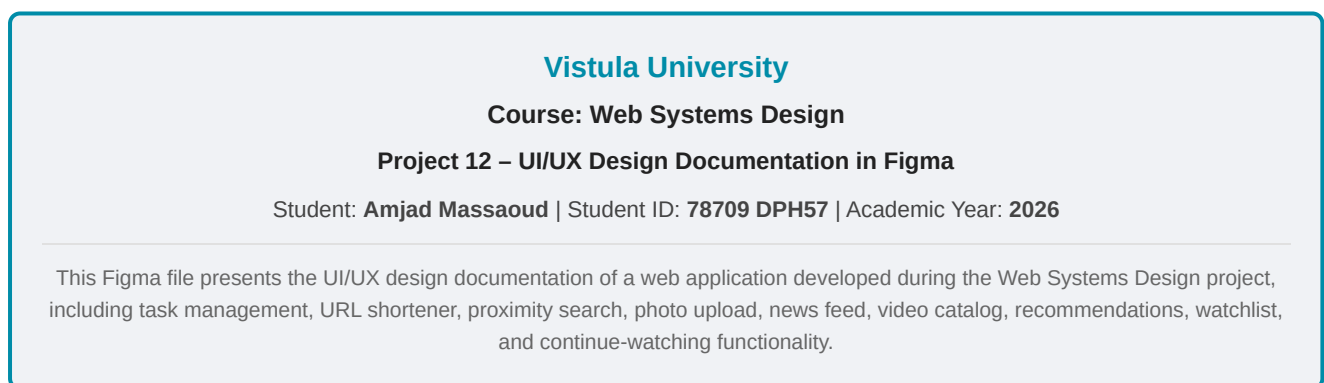


Figure 1. Figma file overview and cover page showing the organized project file, student information, and project description.

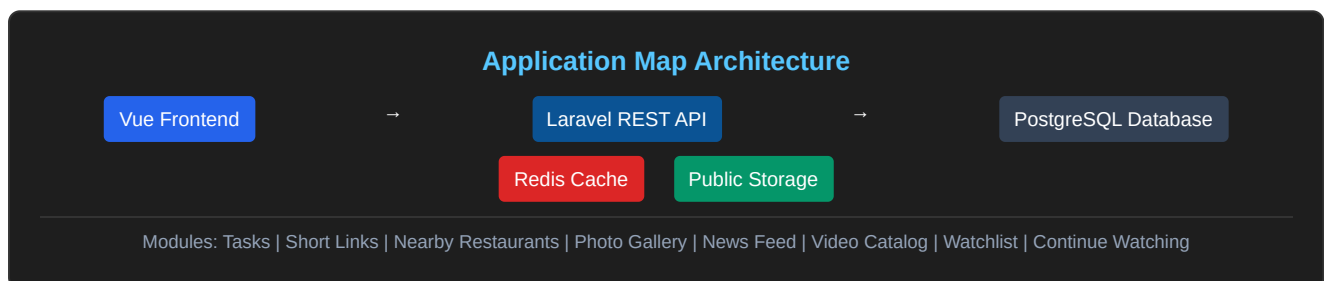


Figure 2. Application map showing frontend, Laravel REST API, PostgreSQL, Redis cache, file storage, Docker Compose services, and application modules.

Figure 3. Design system page showing reusable UI rules, colors, typography, spacing, components, card, table row, navigation item, and status badge examples.

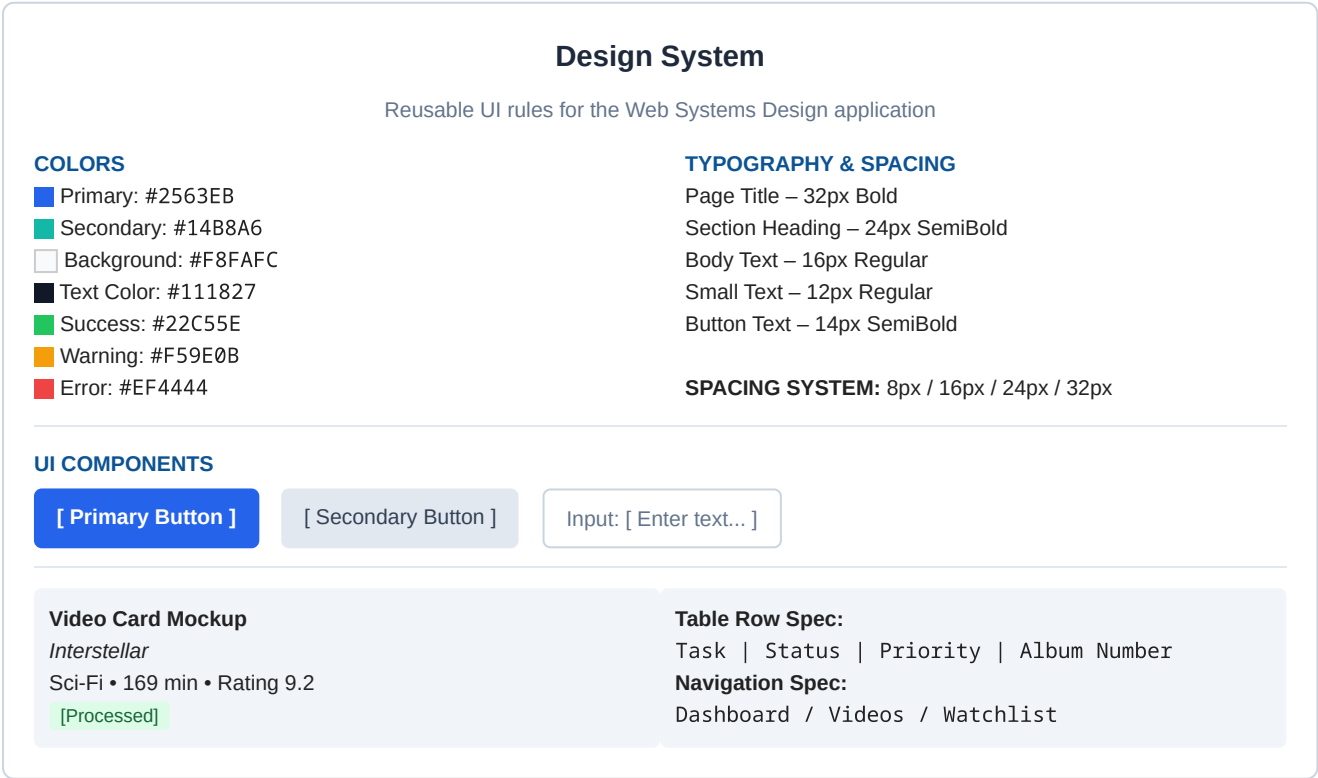


Figure 4. Wireframes page showing low-fidelity sketches for dashboard, task management, URL shortener, photo gallery, news feed, and video catalog.

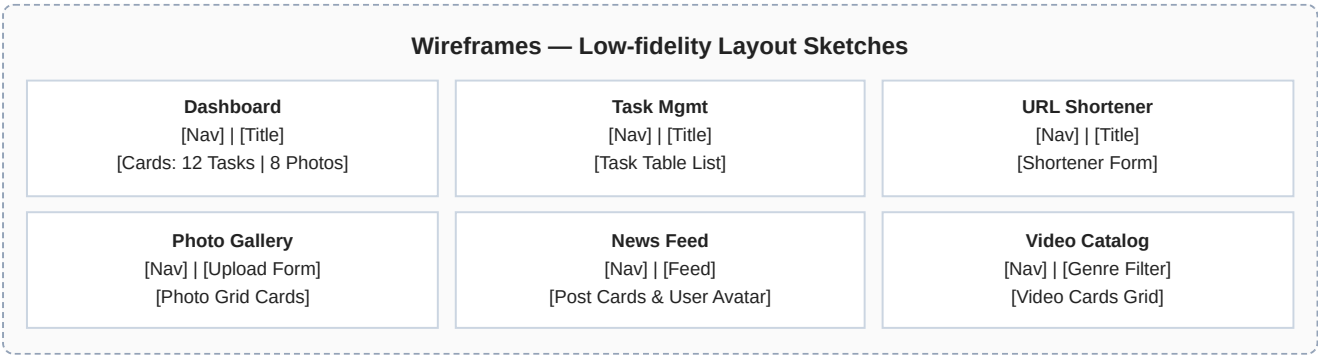


Figure 4. Wireframes page showing low-fidelity sketches for dashboard, task management, URL shortener, photo gallery, news feed, and video catalog.

Figure 5. Final UI screens page showing the dashboard high-fidelity screen with sidebar navigation, status cards, icons, and application data.

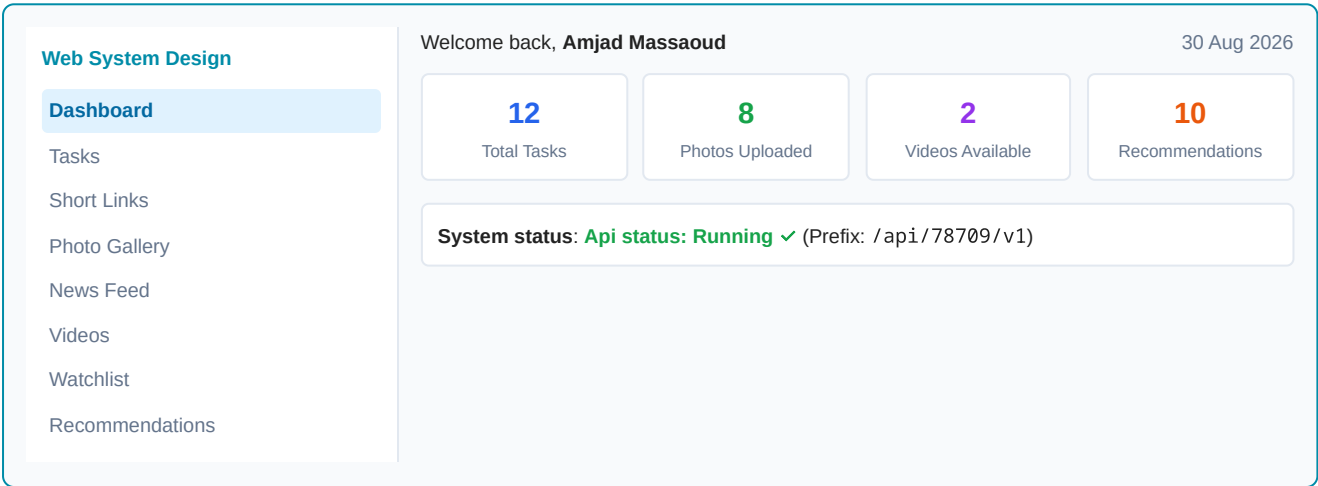


Figure 5. Final UI screens page showing the dashboard high-fidelity screen with sidebar navigation, status cards, icons, and application data.

Figure 6. Prototype flow page showing planned user navigation between dashboard, task management, photo upload, news feed, video catalog, watchlist, recommendations, and continue watching.

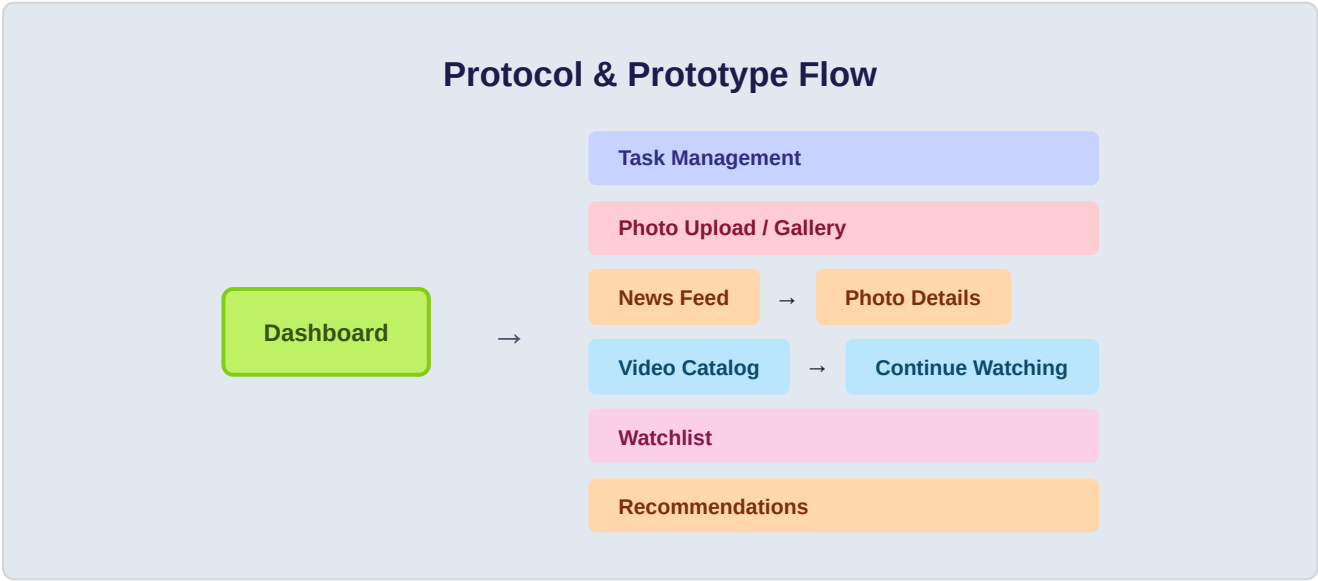


Figure 6. Prototype flow page showing planned user navigation between dashboard, task management, photo upload, news feed, video catalog, watchlist, recommendations, and continue watching.

**Figure 7. API and data mapping table linking screens, user actions, REST API endpoints, HTTP methods, expected responses, and UI states after response.**

| Screen                   | User Action                          | API Endpoint                    | HTTP Method | Expected Response                      | UI State After Response                            |
|--------------------------|--------------------------------------|---------------------------------|-------------|----------------------------------------|----------------------------------------------------|
| <b>Dashboard</b>         | User opens dashboard                 | /api/78709/v1/tasks             | GET         | List of tasks and system data          | Dashboard cards are populated                      |
| <b>Task Management</b>   | User creates a new task              | /api/78709/v1/tasks             | POST        | 201 Created task object                | New task appears in table                          |
| <b>Photo Upload</b>      | User uploads a photo                 | /api/78709/v1/photos            | POST        | 201 Created, photo metadata            | Uploaded photo appears in gallery                  |
| <b>News Feed</b>         | User opens news feed                 | /api/78709/v1/feed              | GET         | 200 OK, feed items from followed users | Feed cards are displayed                           |
| <b>Video Catalog</b>     | User opens video catalog             | /api/78709/v1/videos            | GET         | 200 OK, paginated list of videos       | Video cards are displayed                          |
| <b>Recommendations</b>   | User opens recommendations           | /api/78709/v1/recommendations   | GET         | 200 OK, recommended videos             | Recommended video cards are displayed              |
| <b>Watchlist</b>         | User adds video to watchlist         | /api/78709/v1/watchlist         | POST        | 201 Created, saved watchlist item      | Video appears in watchlist                         |
| <b>Continue Watching</b> | User opens continue-watching section | /api/78709/v1/continue-watching | GET         | 200 OK, unfinished watch history       | Continue-watching cards and progress are displayed |

*Figure 7. API and data mapping table linking screens, user actions, REST API endpoints, HTTP methods, expected responses, and UI states after response.*

**Figure 8. Developer notes page showing reusable components, colors, typography, screen sizes, responsive behavior, API dependencies, known limitations, and future improvements.**

Implementation Relationship & Handoff Notes

**Reusable Components:**

- Primary / Secondary Button
- Text Input / Select Input
- Sidebar / Status Badge
- Video Card / Photo Card

**Typography & Sizes:**

- Title: 32px Bold
- Heading: 24px SemiBold
- Body: 16px Regular
- Desktop: 1440x1024
- Tablet: 768x1024
- Mobile: 390x844

**Responsive & API Rules:**

- Sidebar collapses on mobile
- Cards stack vertically
- Backend: /api/78709/v1
- Caching: Redis 60s TTL
- Storage: public disk

*Figure 8. Developer notes page showing reusable components, colors, typography, screen sizes, responsive behavior, API dependencies, known limitations, and future improvements.*

**Figure 9. API states page showing loading, success, and error states for selected API-based modules.**

| Module    | Loading      | Success      | Error     |
|-----------|--------------|--------------|-----------|
| Videos    | Loading...   | Videos shown | Failed    |
| Photos    | Uploading... | Uploaded     | Too large |
| Watchlist | Saving...    | Added        | Duplicate |

*Figure 9. API states page showing loading, success, and error states for selected API-based modules.*

## 5. Limitations and Future Improvements

The current design is suitable as a minimum complete submission for Project 12. The main limitation is that the final UI screens focus mostly on desktop layout, and not every module has a fully detailed final screen. Some prototype flows are represented visually with arrows instead of full detailed clickable screens.

Future improvements include adding complete mobile screens, more detailed video pages, login and register screens, empty states, dark mode variables, advanced filtering, better accessibility, and a separate Dev Mode inspection screenshot.

## 6. Conclusion

Project 12 improved the Figma design file by adding API states, prototype flow, API mapping, developer notes, and evidence screenshots. The design connects the user interface with the Laravel REST API and documents how the system should behave during loading, success, and error states. The resulting Figma file can be used as a design and implementation handoff document.

## 7. References

1. Figma Help Center: <https://help.figma.com/hc/en-us>
2. Figma Auto Layout documentation: <https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout>
3. Figma Variables documentation: <https://help.figma.com/hc/en-us/articles/15339657135383-Guide-to-variables-in-Figma>
4. Figma Dev Mode documentation: <https://help.figma.com/hc/en-us/articles/15023124644247-Guide-to-Dev-Mode>

## Project 12 – Git Repository & Version Control Verification

All project modules, backend controllers, models, database migrations, automated test suites, frontend components, and compiled assignment reports have been committed and synchronized with the remote GitHub repository:

<https://github.com/Jadtales/uni-projects> (branch master).

```
jad@fedora-2:~/uni-project$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

```
jad@fedora-2:~/uni-project$ git log -1 --oneline
96dbd38 (HEAD -> master, origin/master) - projects
```

